AgentForge: Deterministic Gates for an LLM Multi-Agent Pipeline from Product Idea to Deployed Web Application

***Abstract*—Large language models can now write most of a web application, yet the agent's own report that the work is finished remains the weakest evidence in the process. This paper presents AgentForge, a desktop system in which five LLM agent roles (requirements, prototype, build, quality assurance and deployment) carry a product idea through an adaptive interview, an approved plan, a requirements specification with eleven kinds of diagrams, wireframes, an interactive prototype, a phased build, browser-level testing and a cloud deployment. The roles share one tool-calling engine and hand work to each other through a project record on disk. Every hand-off is checked by code rather than by a model: a depth floor on the plan; schema, plan-coverage and evidence audits on the specification; a named-page gate on the build; requirement verdicts computed from test-runner output; and a probe of the deployed HTTPS address that the deploying agent does not control. Personal data typed by the user is replaced with placeholders before every model request and restored in the generated artifacts. Measurements taken during development show three effects. Writing the specification as five parallel parts cut its generation for a 31-page store from 546 s to 141 s. One conversation carried through a build grew to 900,000 tokens and made each model call about four times slower, which moved the hand-off from the conversation to files. Computing requirement status from test results corrected a report of 0 of 116 requirements met after every test had passed. A protocol for a controlled evaluation is specified.**

***Index Terms*—autonomous software engineering, large language models, multi-agent systems, requirements engineering, software testing, deployment automation, data privacy**

I. Introduction

Ask one model to build a thirty-page web application in a single conversation and it tends to return a smaller one. In AgentForge's early runs the model said as much: partway through, it decided it "must be honest about capacity" and finished a coherent subset of the approved pages. No file it wrote was wrong. The application was simply smaller than the one the user had approved, and only a count against the plan could show by how much.

Code-trained language models have brought function- and module-level programming within reach of automation [1], [2], and agentic systems now resolve a measurable share of real repository issues end to end [3], [4]. The hard part has moved. It is the distance between code that runs and an application someone can trust: the requirements were understood, every promised screen exists, tests show the behaviour the user asked for, and the deployed address answers. Surveys of LLM-based multi-agent systems for software engineering name quality assurance and evaluation as the open problems [5].

AgentForge is built on one rule: a model may write anything, but code decides whether a stage is finished. Five agent roles take turns on one tool-calling engine. They pass work to each other through files in the project's own record, and every stage boundary has a check that reads those files and either lets the project through or names what is missing. The model is used where judgement is needed (choosing the next interview question, writing a page, fixing a failing journey) and kept out of the decision about whether that work is done. The paper makes four contributions:

1) A pipeline from a one-sentence idea to a live HTTPS address in which five agent roles share one engine and one on-disk project record (Section III).
2) Deterministic checks at each hand-off, including requirement status computed from test-runner output rather than from the agent's report (Section III-E).
3) A privacy layer that keeps typed personal data out of model requests while the artifacts keep the real values (Section III-G).
4) Measurements of where a long-running agent pipeline broke and what fixed each break, with a protocol for a controlled evaluation (Section V).

II. Related Work

A. Code Generation and Repository Agents

HumanEval-style benchmarks measure whether a model can turn a docstring into a working function [1]; SWE-bench asks whether an agent can resolve a real GitHub issue [3]. Open-weight families such as Qwen2.5-Coder [2] and Gemma 3 [6] run on a single workstation, which matters for a tool that must work with a locally hosted model. ReAct set the pattern of interleaved reasoning and tool calls that most coding agents follow [7]. SWE-agent showed that the interface an agent is given changes how often it succeeds [4], and Agentless reached results competitive with open-source agents on SWE-bench Lite with a fixed localise, repair and validate pipeline [8]. AgentForge takes the second lesson further: the stage order and every stop condition are fixed in code, and the agent loop runs only inside a stage.

B. Multi-Agent Software Engineering

Dividing a problem among communicating specialists predates language models [9], [10]. The LLM-based versions differ mainly in how agents coordinate. ChatDev runs a virtual software company through chat-chained waterfall phases [11]. MetaGPT encodes standard operating procedures as roles that exchange structured documents [12]. AutoGen frames collaboration as programmable multi-agent conversation [13], and AgentCoder separates a programmer agent from a test designer and a test executor, showing that independently written tests improve the final code [14]. As described in their papers, these systems start from a requirement given as text and end with source code. Interviewing the user, holding the result to a plan the user approved and carrying the application to a deployed address are outside their scope (Table I).

TABLE I. CAPABILITIES AS DESCRIBED IN EACH SYSTEM'S PUBLICATION

Capability	ChatDev [11]	MetaGPT [12]	AutoGen [13]	Agent-Coder [14]	Agent-Forge
Role-specialised LLM agents	Yes	Yes	Yes	Yes	Yes
Interviews the user for requirements	No	No	No	No	Yes
Specification checked by code against an approved plan	No	Partial	No	No	Yes
Progress decided by executed tests	Partial	Partial	Partial	Yes	Yes
Browser journeys and screenshot review	No	No	No	No	Yes
Deployment verified at a public address	No	No	No	No	Yes
Personal data hidden from the model	No	No	No	No	Yes

Partial: MetaGPT writes structured requirement and design documents but does not check them against a plan the user approved; ChatDev, MetaGPT and AutoGen run generated code and feed errors back without a pass criterion fixed in advance.

C. Self-Checking and Long Contexts

Self-Refine loops a model over its own critique of one output [15], and Reflexion turns failure feedback into verbal memory across attempts at a task [16]. Both rely on the model to judge its own work. Without external feedback, models do not reliably correct their own reasoning [17], and LLM judges show position, verbosity and self-enhancement biases [18]. Long inputs bring a separate cost: models use information in the middle of a long context less well than information at its ends [19]. AgentForge keeps model critique where a picture or a document must be read, but the pass or fail decision is always a function of files on disk.

D. Requirements Engineering

ISO/IEC/IEEE 29148 lists the properties of a well-formed requirement, among them being necessary, unambiguous, singular and verifiable [20]. Public corpora such as PURE [21] and the expanded PROMISE set [22] make requirement quality measurable, and multi-agent LLM pipelines have been used to derive UML models from natural-language requirements [23]. AgentForge checks a subset of these properties mechanically and draws its diagrams from the specification it has already checked (Section III-C).

III. SYSTEM DESIGN

AgentForge runs on the user's computer. A Studio interface talks to a Python backend, the backend keeps one engine per project, and the engine runs whichever agent role the current stage needs. Fig. 1 shows the parts and the checks between stages.

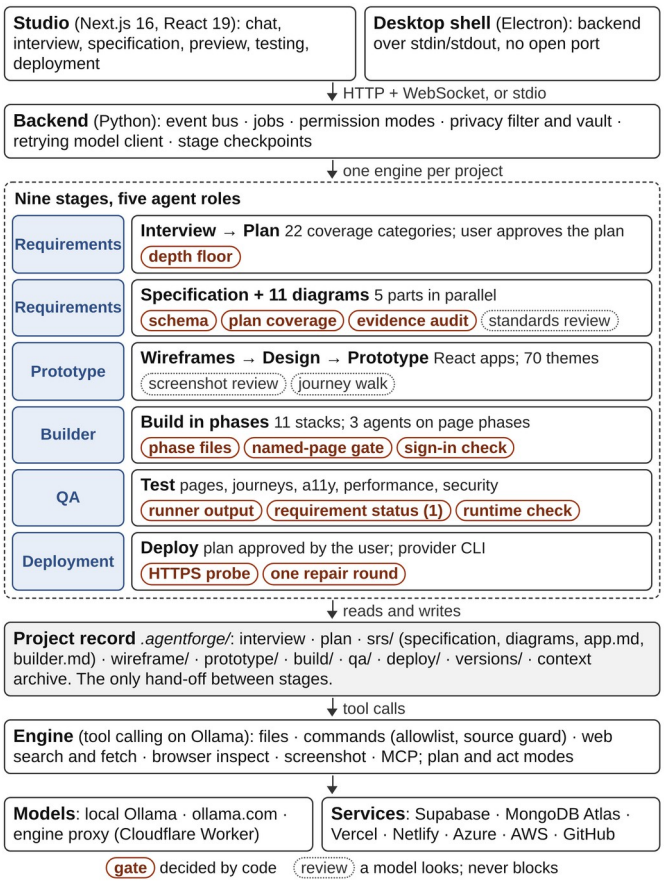


Fig. 1. AgentForge architecture. Five agent roles share one engine and hand off through the project record; the checks between stages are decided by code.

A. Architecture

The pipeline has nine stages: interview, plan, specification, wireframes, design, prototype, build, test and deploy. Five agent roles own them (Table II). A role is not a separate process or a separate model. It is a set of prompts, a stage of the project and a label on the events the Studio shows. All five run on the same engine against the same workspace, and every instruction a model receives is a Markdown file under *prompts/*, so changing how a stage behaves means editing a file, not the code.

Each project has a record folder, *.agentforge/*, beside the application it builds. The interview, the plan, the specification, the wireframe and prototype apps, the build's phase state, the test runs and the deployment record live there. A later stage reads what an earlier one wrote, the Studio's views read the same files, and a stopped stage resumes from its saved state.

TABLE II. AGENT ROLES, WHAT THEY PRODUCE AND WHAT CHECKS THEM

Role	Produces	Checked by code
Requirements	Interview, plan, specification, 11 diagrams, hand-off files, wireframes	Depth floor; schema; plan coverage; evidence audit
Prototype	Theme choice, prototype app, demo account per role	Every screen has a page; app bundles
Builder	Application, unit and journey tests, build report	Phase files; named-page gate; sign-in page
QA	Page and journey fixes, a11y, performance and security results	Runner output; Eq. (1); runtime check
Deployment	Plan the user approves, release, run record	HTTPS probe by the Studio; one repair

B. Engine and Context

The engine is a terminal coding agent built on Ollama's tool-calling API [24]. It exposes ten tools: list, read and search files; write a file or replace one exact span; run a command; search and fetch the web; inspect a local page in a headless browser; and take a screenshot that a vision-capable model is then shown. More tools can be attached through the Model Context Protocol [25]. Any Ollama model with tool calling can drive it, locally or on ollama.com.

A stage runs in two modes. In plan mode the write and command tools are withheld and the model produces a numbered plan. In act mode it carries the plan out and must end with an explicit marker: complete or blocked, and, during an audit, verified or incomplete. The runtime bounds the loop at eight execution rounds, three audit rounds and three consecutive rounds without a tool call, after which the stage is declared blocked. When a step needs a JSON answer, code validates it and sends the validator's message back in the same conversation, up to three times.

The first design kept one conversation per project so that every stage could see everything before it. A compaction step summarised older turns once the conversation passed 88% of the model's window, four stretches at a time, with a mechanical digest of file paths, commands and requests as the fallback when the model returned no summary. Section V-C shows why that was not enough. Each stage, and each build phase, now starts from a small conversation: earlier turns are archived verbatim, a short memory is kept, and the stage reads what it needs from the record.

C. Requirements: Interview, Plan and Specification

The interview asks one question at a time. The first, which kind of product this is, comes from a catalogue of ten product types and costs no model call. Every later question is chosen by the model from what is still unknown across 22 requirement categories (purpose, roles, workflows, data, permissions, business rules and so on), each tracked as known, partial, unknown or not applicable. A one-screen tool finishes in a handful of questions and a multi-role system takes more, up to a ceiling of 25, after which the model writes a summary for the user to confirm or correct. A user who skips the interview gets a plan that states every assumption it made.

The plan is what the user approves, and it is a ceiling: the specification's pages come from its screens, its tables from its records and its roles from its users. A depth floor rejects a plan that cannot be built from. The product intent must be a full paragraph of at least 120 characters, and every screen needs a purpose. In a record-based application every record keeps at least two fields and at least one workflow has two or more steps. If the product has sign-in, at least one role needs two things it can do, the plan must say how accounts are created, and open sign-up may not create a privileged role such as an administrator, manager or cashier.

The specification is written in five parts at once (frame, tables, API, requirements and quality) from page routes and table names computed once from the plan, then joined and validated against a Pydantic schema that requires at least three functional and three non-functional requirements. The *coverage gate* then copies in what the plan states and the draft omitted, and counts what is still missing: a planned screen with no page (routes compared without letter case or parameter names), a record with no table, a feature, workflow or role ability with no requirement, and a functional requirement with no traceability row. Any remaining gap stops the stage.

Two quality checks follow. The *evidence audit* is seven fixed rules: both requirement sets exist; identifiers follow FR-### or NFR-### and are unique; each requirement says "shall"; each names a verification method from a fixed list; each functional requirement has a rationale; each non-functional requirement carries a number with a unit or a named standard; and each functional requirement has exactly one traceability row with a test case. A model reviewer then scores functional, non-functional, security, ambiguity and traceability quality from 0 to 5 and lists findings. A finding that cites a requirement not in the document is rejected, every failed audit rule is added as a blocker, and Python, not the reviewer's verdict, decides that the draft passed: no blocker and every score at least 3. Repairs are JSON edits applied only if the result still validates. By default one review round runs and its outcome is stamped on the document, because asking a model to rewrite a large specification risked losing content.

Eleven diagram kinds are then drawn in Mermaid, each in its own call and six at a time: system context, use case, entity-relationship, sequence, activity, class, state machine, data flow, BPMN, component and deployment. Each is rendered to SVG, photographed and reviewed by a vision model for clipped labels, crossing lines and unreadable scale, for at most two rounds; a diagram that comes back worse is restored. Two hand-off files close the stage. Both are projections of the specification made by code, since a document a model writes can be truncated. *app.md* holds what the user asked for, in their words, and the site map; *builder.md* holds the full specification for the build, the tests and the deployment.

D. Wireframes, Design and Prototype

The wireframes and the prototype are both small React applications (Vite, TypeScript, Tailwind CSS and shadcn/ui) that the agent writes with its own file tools, following Anthropic's web-artifacts-builder skill [26]. The Studio creates each app and bundles it into one self-contained HTML page; the pages are the agent's. Wireframes are drawn from *app.md* alone, so they show what the user asked for rather than what the specification inferred. For ten or more screens, one agent writes the shared layout and the first three pages and three agents write the rest in parallel. Three is close to the number of concurrent requests the hosted model service accepts.

The user then picks one of 70 design themes, each a DESIGN.md file the prototype must follow. The prototype is a new high-fidelity app written from the approved wireframes, the theme and *app.md*. Where the product has sign-in, code derives one fictitious account per role from the specification; the prototype signs in with these and the build later seeds the same ones. Two reviews follow. Every page is photographed at 1440 and 390 pixels wide and a vision model lists visible defects, up to 40 of which are fixed. Each user journey (up to 24, of up to ten steps) is then walked in a real browser: the walker signs in as the journey's role, reaches each screen by clicking links the pages actually have, asks the model what to press or fill, and photographs every step. These are reviews, not gates, and are skipped when the model cannot see pictures.

E. Build and Verification

The builder installs one of eleven stack templates (Next.js, Vite or Remix on Supabase or MongoDB, and a MERN microservice layout, with or without a Supabase Storage bucket for uploads) into an empty workspace, never over existing work. A preflight step checks Node.js, a browser and the stack's database before anything is planned, and lists everything missing in one prompt. The model may then ask the user up to five questions it cannot settle itself. After that the build never stops to ask. A decision with no safe default takes the most conservative reading of the specification, a missing credential becomes real code against a documented environment variable, and each such choice is recorded in the build report as a gap labelled *gap*, *unavailable*, *untested* or *known*.

The Studio, not the model, holds the list of phases (Fig. 2): a foundation (data models, authentication, shared layout), page phases of four approved pages each, a wiring phase with the seed data, and the unit and journey tests. Once the foundation stands, three agents build page phases in parallel and the two test phases run together. Each phase starts a fresh conversation that begins with the build request. A phase is finished when its progress record names files that exist and are not empty; otherwise it is asked for again, at most twice. When every phase has run, an approved page that is still not built fails the build by name. A sign-in page that displays demo credentials is rejected as well.

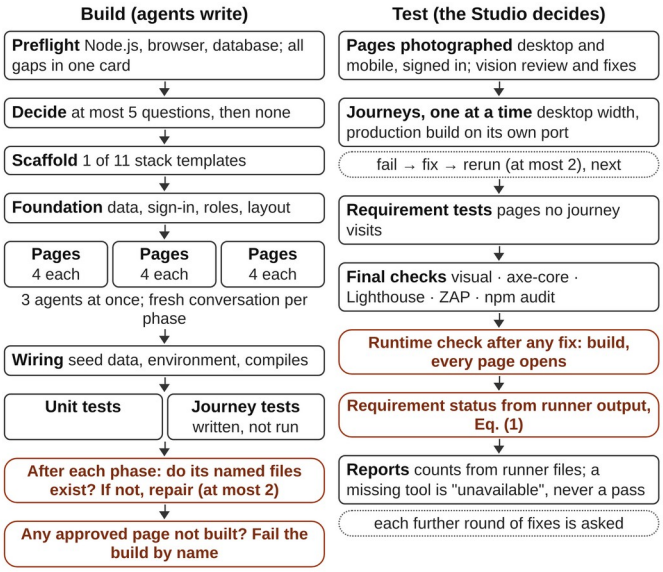


Fig. 2. Build and test sequence. Agents write the application and fix what fails; the Studio decides when a phase, the build and each requirement are done.

Testing follows a fixed order. Every page is photographed whole at desktop and mobile width, signed in as a role that may open it, and reviewed for visible defects. The Studio then runs each user journey on its own at desktop width, through a runner that starts the production build on a free port and, on MongoDB stacks, with a fresh, uniquely named test database that is dropped afterwards. A failing journey is fixed and rerun, at most twice, before the next one runs. Requirements on pages that no journey visits get tests of their own. The agent then runs the remaining layers once each: visual comparison, accessibility with axe-core [27], performance with Lighthouse [28], a ZAP baseline scan [29] (passive checks of its own when ZAP cannot run) and a dependency audit. A fix made at that point is checked with a runtime test (build, then every page opens), not with another journey run. The agent cannot run the whole journey suite itself; the command is refused, because the Studio runs it once and owns the result. A tool that is not available is recorded as unavailable, never as a pass, and every count in the reports is read from the runners' own output files.

Requirement status is computed, not reported. Let $P(r)$ be the pages that the traceability row of requirement r names, $J(p)$ the journeys with a step on page p, and $pass(j)$ true when the desktop test of journey j passed. When no test names r in its title,

$$\mathrm{met}(r) \;\Leftrightarrow\; P(r) \neq \varnothing \;\wedge\; \forall p \in P(r)\;\; \exists j \in J(p) : \mathrm{pass}(j) \qquad (1)$$

A requirement is *failed* when some page in $P(r)$ is visited only by failing journeys and *untested* otherwise. When tests do name r, they decide on their own. The result overwrites whatever the builder wrote and is marked as decided by the Studio.

F. Change Requests and Deployment

After the build, a request typed into the chat is planned first. The planner reads the project, may ask up to three questions and shows a plan the user can approve, revise or cancel. Each approved change becomes a version (v1, v2, ...) that lists the files it touched and the test results before and after. The user can also point at elements in the live preview, which are described to the agent by tag, text, attributes and markup.

Deployment uses the same plan-first flow. The targets are Vercel, Netlify, Azure App Service, AWS EC2, AWS ECS and GitHub, filtered by what the stack supports. The agent works through each provider's own command-line tool, signed in through that tool's browser or device-code login, so no token is pasted. Its rules favour one pass over repetition: no test is rerun, the application is built once, at most three questions are asked, and the smoke proof is at most six read-only requests. When the agent records the run as live, the Studio checks the result itself. It requests the public address and every check the agent recorded, and rejects an address that is not HTTPS or whose host is local or a private IP, and any status outside the expected range; 429 and gateway errors from a host that is still starting are retried after 5 s and 10 s. A failure goes back to the agent once. If the address still does not hold, the run ends as failed, with the cause.

G. Safety, Privacy and Reliability

Each project runs in one of four permission modes: automatic (only risky commands are asked about), manual, accept edits, and bypass, which must be switched on first. Commands are checked against an allowlist of 132 program names and a deny list (*sudo*, a download piped into a shell, Invoke-Expression, deleting the root or home directory, a credential file sent to a network command). The code calls this defence in depth, not a sandbox. AgentForge's own source is snapshotted before every command and restored if the command changed it.

Personal data does not reach a model in clear text. Everything the user types passes through Microsoft Presidio [30] with a spaCy English model [31], set to 15 entity types including Sri Lankan identity and phone numbers. Each value found is kept, encrypted, in a per-project vault on the user's computer under a placeholder numbered for the whole project

(<PERSON_1> is the same person in every message). The one client every model call passes through hides known values on the way out and restores them in the answer and in the arguments of tools that act on the local machine; web search, web fetch and plugin tools keep the placeholders. If Presidio is missing, the message is not sent. Text shaped like a key or a connection string is refused in the chat and must be saved in Settings instead.

A transient model-service failure (a reset connection, a timeout, HTTP 408, 425, 429 or a 5xx error) is retried up to ten times with back-off from 2 s to 30 s. Every stage writes a checkpoint when it starts, finishes or fails, so a run cut off by a restart resumes. Hosted models are reached with the user's own ollama.com key or through a Cloudflare Worker that holds the service key, passes through eight API routes and meters calls against the user's account.

IV. Implementation

AgentForge 1.0.3 has about 32,300 lines of Python in the backend, engine and agents, 27,800 lines of JavaScript in the Studio, and 19,500 lines of Markdown prompts, including the design themes and the deployment skill pages. The backend serves its HTTP API and its RFC 6455 WebSocket feed from the Python standard library, and uses Pydantic for the specification schema, the cryptography package for the vault, and Presidio and spaCy for the privacy filter. The Studio is a Next.js 16 and React 19 application. The Windows desktop app wraps it in Electron, where the backend opens no network port and talks to the window over standard input and output; a 150 KB online installer fetches the app, an embedded Python 3.12 and pinned command-line tools, each checked against a SHA-256 manifest.

Generated applications come from 236 template files: the eleven stacks, a shared testing kit (Playwright [32], Vitest, axe-core, Lighthouse CI, ZAP and a server runner) and 18 stack and testing guides that the builder reads before it plans. Screenshots are taken with a locally installed Edge, Chrome or Chromium in headless mode. Models are chosen by the user: the bakery project in Section V-B used gpt-oss:120b served through Ollama's cloud, and accounts default to DeepSeek V4.1 Flash. The repository holds 1,733 Python unit tests and twelve Node test files, which cover the engine, every gate, the privacy filter and the deployment flow with stand-in models instead of live calls.

V. Evaluation and Preliminary Results

A. Research Questions and Protocol

The observations in Section V-C were gathered while the system was being built, so they cannot attribute an improvement to one component. The controlled study is the next step: a fixed suite of application ideas, from a one-screen tool to stores of over thirty screens, built with the same model and settings under each condition of Table III, three runs per condition, reported as mean and standard deviation. Requests are logged at the single model client, which gives exact token counts and makes leaked personal data countable.

TABLE III. Research Questions, Conditions and Metrics

Research question	Condition	Primary metrics
RQ1: Do the coverage gate and audit make the specification more complete?	Gates on vs. first draft kept	Plan items with no requirement; audit score; rater-found omissions
RQ2: Does a phased build deliver every approved page?	Phased vs. one conversation	Pages built of pages approved; time; tokens
RQ3: How far do agent-reported statuses diverge from Eq. (1)?	Builder report vs. Eq. (1) vs. blind manual test	Cohen's κ with the human verdict; false "met"
RQ4: What do per-phase conversations cost or save?	Fresh per phase vs. one shared, compacted	Tokens and latency per call; total time; context defects
RQ5: How reliably does deployment reach a probed live address?	Each app to each allowed target	Success rate; time; repairs; questions asked
RQ6: Does the filter keep personal data out of requests?	Synthetic personal data; filter on vs. off	Values in logged requests; wrongly hidden terms

B. Development Corpus

The observations come from the authors' own projects: a bakery ordering application (interview to prototype), a 28-page application (specification to deployment), a 31-page online store and a store whose plan had 71 screens. Each row of Table IV is one observed run before and, where stated, after a change, as recorded in the version history. The rows explain the current design; they are not a comparison with other systems.

C. Observations

Table IV lists twelve failures in the order of the pipeline. The coverage gate gave the earliest result: on the bakery project it took the first specification draft from one diagram to seven and from three functional requirements to six. Before the changes in rows 4 to 8, the 28-page application took about seven hours of machine time from specification to deployment.

TABLE IV. Failures Observed During Development and the Change Each Led To

Failure observed	Measurement	Change made
1. One conversation builds the whole app	A coherent subset; model cited "capacity"	Numbered phases; named-page gate
2. Interview turn rewrote all facts (~4,000 tokens)	~20 s per question (up to 64 s); after: 1.9–4.6 s	Question alone; facts updated in background
3. Specification as one ~60,000-token reply	546 s, 31-page store; after: 141 s	Five parts in parallel
4. One conversation through all build phases	900,000 tokens; 21 s per call (5 s at 50,000)	Fresh conversation per phase
5. One agent draws all wireframes	About 1.5 h for 117 pages	Three agents after the shared layout
6. One agent builds all pages	2 h 37 min for 31 pages	Three agents on page phases
7. All journeys run, then fixed together	About 1.5 h, 31-page store	One journey at a time
8. Search walked test frames and packages	16 min for one search; deploy plan 37 min	Skip outputs and binaries; 20 s cap
9. Requirements marked before tests ran	0 of 116 met after all tests passed	Status from runner output, Eq. (1)
10. Routes compared with letter case	117 pages for 71 screens	Case-insensitive route keys
11. "Admin", "Require" taken for names	117-page wireframe ended with 2 pages	Names need two name-shaped words
12. Screenshot added to a long conversation	2.5 MB answered; 3.4 MB got HTTP 500	Pictures only below 2.4 MB

D. Discussion

Shared files beat a shared conversation. The first design treated the conversation as the project's memory. It failed on cost, not correctness: by the build every call resent hundreds of thousands of tokens, and a request that also carried a screenshot exceeded what the service accepted. The switch to small conversations was safe because every stage's input was already a file (the plan, the specification, the hand-off files, the per-phase progress records). This fits the evidence that models use long inputs unevenly [19].

Self-reports are the weakest evidence. Rows 1 and 9 of Table IV are one failure in two stages: the model's account of its work differed from the files. Each fix replaced a statement by the model with a function of what is on disk, consistent with the finding that models do not reliably correct themselves without external feedback [17]. The specification reviewer's verdict is recorded but not obeyed for the same reason.

A check that fires wrongly does damage. With "require" taken for a name, the agent read <PERSON_3>("fs") and wrote it back into the code; one capital letter in a route produced 46 duplicate pages. A check that rewrites content needs precision more than recall, and both checks were narrowed until they fired only on the case they were written for.

VI. Limitations and Future Work

Five limitations bound these claims. First, the evidence is internal and uncontrolled: single runs on the authors' projects with no baseline. Second, the coverage gate matches plan items to requirements by shared words, and the evidence audit checks the form of a requirement (an identifier, "shall", a unit), not its truth. Third, Eq. (1) is only as strong as the journeys: a journey that visits a page but asserts little still marks that page's requirements met. Fourth, the screenshot reviews need a vision-capable model, the screenshots it sees carry the real personal data, and name detection uses an English model. Fifth, the system targets JavaScript web applications on Supabase or MongoDB, ships for Windows, and its command checks are not isolation. Next come the controlled study, journey assertions tied to acceptance criteria, and agent commands run in a container.

VII. Conclusion

AgentForge started from the view that the hard part of autonomous software engineering is not writing code but knowing when it is done. It hands that decision to code at every boundary: a depth floor on the plan, coverage and evidence checks on the specification, a named-page gate on the build, requirement status computed from test results, and a deployed address checked by the Studio rather than by the agent that deployed it. Building it showed two things the first design did not expect: a conversation is a poor place to keep a project, and every place where a model's report stood in for evidence eventually gave a wrong answer that a short function over the files could correct. The study of Section V-A will test whether this holds beyond the projects the system was built on.

References

[1] M. Chen et al., "Evaluating large language models trained on code," arXiv:2107.03374, 2021.

[2] B. Hui et al., "Qwen2.5-Coder technical report," arXiv:2409.12186, 2024.

[3] C. E. Jimenez, J. Yang, A. Wettig, S. Yao, K. Pei, O. Press, and K. Narasimhan, "SWE-bench: Can language models resolve real-world GitHub issues?" in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2024.

[4] J. Yang, C. E. Jimenez, A. Wettig, K. Lieret, S. Yao, K. Narasimhan, and O. Press, "SWE-agent: Agent-computer interfaces enable automated software engineering," in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, 2024.

[5] J. He, C. Treude, and D. Lo, "LLM-based multi-agent systems for software engineering: Literature review, vision, and the road ahead," *ACM Trans. Softw. Eng. Methodol.*, 2025, doi: 10.1145/3712003.

[6] Gemma Team, "Gemma 3 technical report," arXiv:2503.19786, 2025.

[7] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao, "ReAct: Synergizing reasoning and acting in language models," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2023.

[8] C. S. Xia, Y. Deng, S. Dunn, and L. Zhang, "Agentless: Demystifying LLM-based software engineering agents," arXiv:2407.01489, 2024.

[9] S. Russell and P. Norvig, *Artificial Intelligence: A Modern Approach*, 4th ed. Hoboken, NJ, USA: Pearson, 2021.

[10] M. Wooldridge, *An Introduction to MultiAgent Systems*, 2nd ed. Chichester, U.K.: Wiley, 2009.

[11] C. Qian et al., "ChatDev: Communicative agents for software development," in *Proc. 62nd Annu. Meeting Assoc. Comput. Linguistics (ACL)*, 2024.

[12] S. Hong et al., "MetaGPT: Meta programming for a multi-agent collaborative framework," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2024.

[13] Q. Wu et al., "AutoGen: Enabling next-gen LLM applications via multi-agent conversation," arXiv:2308.08155, 2023.

[14] D. Huang, J. M. Zhang, M. Luck, Q. Bu, Y. Qing, and H. Cui, "AgentCoder: Multi-agent-based code generation with iterative testing and optimisation," arXiv:2312.13010, 2023.

[15] A. Madaan et al., "Self-Refine: Iterative refinement with self-feedback," in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, 2023.

[16] N. Shinn, F. Cassano, A. Gopinath, K. Narasimhan, and S. Yao, "Reflexion: Language agents with verbal reinforcement learning," in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, 2023.

[17] J. Huang, X. Chen, S. Mishra, H. S. Zheng, A. W. Yu, X. Song, and D. Zhou, "Large language models cannot self-correct reasoning yet," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2024.

[18] L. Zheng et al., "Judging LLM-as-a-judge with MT-Bench and Chatbot Arena," in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS), Datasets and Benchmarks Track*, 2023.

[19] N. F. Liu, K. Lin, J. Hewitt, A. Paranjape, M. Bevilacqua, F. Petroni, and P. Liang, "Lost in the middle: How language models use long contexts," *Trans. Assoc. Comput. Linguistics*, vol. 12, pp. 157–173, 2024.

[20] *Systems and Software Engineering—Life Cycle Processes—Requirements Engineering*, ISO/IEC/IEEE 29148:2018, 2018.

[21] A. Ferrari, G. O. Spagnolo, and S. Gnesi, "PURE: A dataset of public requirements documents," in *Proc. IEEE 25th Int. Requirements Eng. Conf. (RE)*, 2017, pp. 502–505.

[22] M. Lima, V. Valle, E. Costa, F. Lira, and B. Gadelha, "Software engineering repositories: Expanding the PROMISE database," in *Proc. 33rd Brazilian Symp. Softw. Eng. (SBES)*, 2019.

[23] P. Giannouris, "Multi-agent LLM system for UML class diagram generation from natural language requirements," arXiv:2511.22409, 2025.

[24] Ollama. [Online]. Available: https://ollama.com

[25] Anthropic, "Model Context Protocol specification." [Online]. Available: https://modelcontextprotocol.io

[26] Anthropic, "web-artifacts-builder," Agent Skills repository. [Online]. Available: https://github.com/anthropics/skills

[27] Deque Systems, "axe-core: Accessibility engine for automated web UI testing." [Online]. Available: https://github.com/dequelabs/axe-core

[28] Google, "Lighthouse." [Online]. Available: https://developer.chrome.com/docs/lighthouse

[29] ZAP by Checkmarx (formerly OWASP ZAP), "Zed Attack Proxy." [Online]. Available: https://www.zaproxy.org

[30] Microsoft, "Presidio: Data protection and de-identification SDK." [Online]. Available: https://github.com/microsoft/presidio

[31] M. Honnibal, I. Montani, S. Van Landeghem, and A. Boyd, "spaCy: Industrial-strength natural language processing in Python," 2020, doi: 10.5281/zenodo.1212303.

[32] Microsoft, "Playwright." [Online]. Available: https://playwright.dev